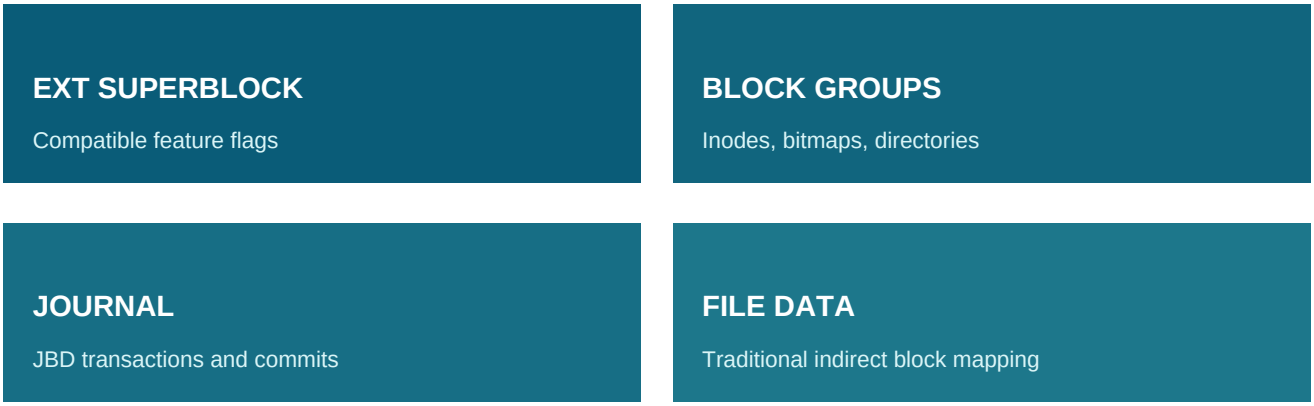
EXT3 - journalled extended filesystem

HISTORY / ORIGINAL DESIGN / DISK FORMAT

The ext2-compatible journalling evolution, implemented in the Linux kernel to reduce crash-recovery time and preserve metadata consistency.

This is a technical reference to the upstream filesystem's historical design, not an installer manual. The format's on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

EXT3 - journalled extended filesystem

HISTORICAL CHRONOLOGY

Ext3 was developed by Stephen Tweedie in the late 1990s, with kernel documentation identifying a September 1999 release. The project built on ext2 instead of introducing an incompatible new layout. Its journalling changes entered mainline Linux in the 2.4-era, helping Linux systems avoid the expensive whole-volume consistency scans that could follow a crash on an ext2 installation.

The conversion story was central to adoption: ext3 can add a journal to an ext2-formatted volume without wholesale copying every file. The resulting shared heritage made it practical for distributions to move to journalling while retaining recovery tools and established ext2 metadata structures. Ext4 later superseded ext3 for many modern workloads while retaining support for a compatible subset.

EXT3 - journalled extended filesystem

ORIGINAL ON-DISK ARCHITECTURE

An ext3 filesystem retains ext2's primary superblock, block groups, descriptors, allocation bitmaps, inode tables and file-block mapping. The HAS_JOURNAL compatible feature flag indicates a journal. The journal commonly resides in a specially designated inode within the filesystem, although external journals are possible. Its block format, headers, descriptors and commit records are part of the JBD journalling subsystem.

Ext3 directory entries still refer to inodes; inodes use familiar direct and indirect pointers rather than ext4's standard extent-tree layout. Revision and extension bits matter: a driver should inspect explicit feature flags, not rely on a name string or assume all ext-family layouts are interchangeable. JBD journal blocks have their own endianness and transaction rules.

EXT3 - journalled extended filesystem

ALLOCATION AND DIRECTORIES

Ext3 uses block and inode bitmaps for each group, with locality-aware allocation policy inherited from ext2. Regular file data is mapped through inode pointers and indirect blocks. Directory entries contain inode numbers, record lengths, name lengths and bytes of the name. Larger directory optimisations, including HTree indexing where supported, change lookup behaviour without abandoning the fundamental inode namespace.

Hard-link counts, metadata ordering and atomic rename semantics must be read together with journal operation. A directory tree is not recovered merely by seeing a committed journal record; all referenced blocks and feature-specific metadata must be interpretable and within bounds. The journalling layer protects an ordered set of modifications rather than replacing all underlying format constraints.

EXT3 - journalled extended filesystem

INTEGRITY AND RECOVERY

JBD logs transactions that can be replayed after a crash. The familiar write modes differ: journal mode can journal file data as well as metadata, ordered mode orders data relative to associated metadata commits, and writeback provides weaker data ordering. It is incorrect to say ext3 always journals every file byte. A successful journal replay repairs many incomplete metadata transactions, but cannot replace checks of damaged sectors or corrupted inode maps.

The recovery process identifies committed transactions and replays appropriate metadata updates before presenting the filesystem as consistent. If journal structures are torn or corrupt, e2fsck remains necessary. Write barriers and stable-storage assumptions are essential to interpreting a commit as durable. Journal existence does not protect against malicious or concurrent offline modification.

EXT3 - journalled extended filesystem

DESIGN RATIONALE AND LIMITATIONS

The ext3 approach prioritised backward compatibility, conservative semantics and shorter restart time after power loss. This came at the cost of extra journal I/O and continued dependence on older block-pointer mapping. Ext4’s extent mapping, larger-volume support and delayed allocation were designed to address workloads ext3 could not serve efficiently.

A journal is not a backup, immutable snapshot or comprehensive protection of user data. Filesystem capacity and directory limits depend on selected geometry and revision; a modern driver that mounts ext3 must honour the supported feature masks and carefully distinguish ext3 metadata from ext4-only extensions.

EXT3 - journalled extended filesystem

INFILTRATOROS INTEGRATION BOUNDARY

EXT3.so may share source architecture with the InfiltratorOS EXT4 project, but an ext3 volume remains its own on-media identity. Build-system reuse does not justify enabling unsupported ext4 features or assuming ext4's full API and recovery path exist in the ext3 module.

The filesystem's original journal and Linux recovery model are separate from any InfiltratorOS application-level defragmentation journal. Linux '.ko' interoperability, source-first builds and Boot usage require explicit evidence beyond a compiled PDF.

EXT3 - journalled extended filesystem

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. Linux kernel: Ext3 Filesystem history and design

https://www.kernel.org/doc/html/v6.8/filesystems/ext3.html

2. Stephen C. Tweedie, ext3 development announcement, 2001

https://lwn.net/2001/0111/a/ext3.php3

3. Linux ext4 journal format notes

https://docs.kernel.org/filesystems/ext4/journal.html

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

EXT3 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Begin with the ext-family superblock and feature flags. Ext3 retains the familiar block groups, inode tables and directory structures of ext2.

2. Locate the journal, normally associated with a reserved inode or an external journal device, and confirm its metadata before replay.

3. Interpret journal descriptor, commit and revoke records using JBD’s record format. An incomplete or uncommitted transaction must not become live metadata.

4. Distinguish ordered, writeback and journalled data modes; they express different guarantees about user data versus metadata updates.

5. Apply metadata replay to a consistent ext filesystem image before relying on bitmaps or directories. A clean-looking inode table alone is not enough.

6. Resolve inode data through the block pointer mapping appropriate to its feature set; ext3 does not automatically imply ext4 extent trees.

7. Read file and directory records using the versioned ext format, including directory index features when enabled.

8. Keep external journal identity and feature compatibility separate from ordinary block-device addresses; a missing external journal changes recovery possibilities.

EXT3 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Confirm journal UUID, sequence ordering and transaction boundaries before recovery. Journal records from a different volume must never be replayed.

2. Account for revoke records so stale logged blocks are not restored over newer transactions.

3. Respect the distinct guarantees of journalling modes: metadata consistency is not equivalent to durable contents for every recently written file.

4. Verify that replayed metadata remains within filesystem and block-group bounds and that the resulting inode, bitmap and directory state is coherent.

5. Do not interpret every ext3 volume as ext2 simply because the basic inode layout looks familiar; an outstanding recovery flag matters.

6. After replay, check incompatible and read-only feature masks before writable access, particularly when using a driver from a different era.

7. Preserve the journal and the original image when an unclean shutdown is suspected; uncontrolled fsck can destroy useful recovery evidence.

8. Distinguish historical ext3/JBD structures from ext4/JBD2 additions rather than borrowing more recent feature definitions indiscriminately.